

Golem-One XL

From next-token prediction to System One decisions

Model card: every part we built in Go, the papers each part follows,
and how a text generator became a decision model

The Golem project
Technical Report 11

September 2026

Abstract

Golem is a GPT-style language model written from scratch in Go, one layer at a time, with no machine-learning framework. Each layer follows the research paper that introduced it and is checked against it. The tokenizer, attention, normalization, feed-forward network, backpropagation, optimizer, preference training, a hand-written ARM64 matrix kernel and a CUDA training engine are all our own code.

Golem-One XL is where that work leads. It is a 1.56-billion-parameter Golem, the shape of the largest GPT-2, initialized from OpenAI’s released GPT-2 XL weights. It no longer answers by writing text one token at a time. It answers *typed questions* with a probability for every allowed answer, all questions in one pass, and can never give an answer that was not offered.

This card tells the whole story for a general technical reader:

1. how the token-by-token model was built, part by part, and which paper each part follows;
2. why generating text is a poor way to make decisions, and where the “System One” idea came from;
3. how the same transformer, read differently, becomes a decision model;
4. how a 1.56B model was trained on two free GPUs;
5. what the model is for, what it is not for, and what has not been measured yet.

Contents

1	The model at a glance	3
2	The story in brief	3
3	Part I — Building a token-by-token model in Go	3
3.1	The architecture	3
3.2	Each part, the paper it follows, and how it was checked	4
3.3	Making it fast	5
3.4	Teaching it to converse	6
4	Part II — From token by token to System One	6
4.1	Why generating text is a poor way to decide	6
4.2	Where the idea came from	6
4.3	Golem-One: the same network, read differently	7
4.4	Why read differently instead of building a new network	8

5	From GPT-2 XL to Golem-One XL: what we changed	8
6	Part III — Training Golem-One XL	10
6.1	Why the largest size	10
6.2	What it learns from	11
6.3	Training so far	12
6.4	Honest probabilities	12
7	Learning from its own decisions	12
8	Design choices	13
9	Intended use and limitations	13
10	Lineage: where each piece comes from	14
11	Using it	14

1 The model at a glance

Name	Golem-One XL
Kind	Decision model (“System One”): a text state and typed questions in, a probability for every declared answer out
Built with	Go, standard library only; Go assembly for the ARM64 kernel; CUDA C++ for the GPU engine
Network	<code>golem-xl</code> : 48 pre-LN transformer blocks, width 1,600, 25 attention heads, 1,024-token context, GELU, output layer tied to the embedding; 1,557,611,200 parameters
Tokenizer	Byte-level BPE with GPT-2’s 50,257-token vocabulary, reimplemented in Go
Starting weights	OpenAI GPT-2 XL, loaded into Golem’s own engine
Adaptation	LoRA rank 16 on every linear layer, plus LayerNorm and bias terms: 20.7M trained parameters (1.33%)
Training data	549,223 human-labeled decision questions from 28 open datasets in 13 skills, 39,740 human explanations, 60,000 Snake game positions
Hardware	Two NVIDIA T4 GPUs (Kaggle free tier)
Status	Training in progress: 37,224 of 81,122 steps (46%) on 25 September 2026
Language	English

Table 1: Golem-One XL in one table.

2 The story in brief

The project began with one diagram of a decoder-only transformer (Figure 1) and a rule: build every box of it in Go by hand, in order, and write a research-style report for each. Ten reports later the diagram is complete. It runs fast on a laptop and on GPUs, it converses, and it has learned to decide.

Report	Milestone	What it added
1	The input layer	Byte-level BPE tokenizer, token embeddings, positional encoding
2	Causal attention	Masked multi-head self-attention with a hand-derived backward pass
3	Layer normalization	LayerNorm, residual connections, GPT-2’s pre-LN block
4	Feed-forward network	GELU, the 4× FFN, a complete block; parameter count equal to GPT-2 small
5	The output head	N blocks, final LayerNorm, tied head, softmax, cross-entropy, full backward pass
6	A faster engine	NEON assembly kernel (112 GFLOP/s per core), CUDA backend
7	Conversation	Supervised fine-tuning and DPO preference training
8	Looking ahead	A comparison with Kimi K3’s linear attention
9	Golem-One	Typed, calibrated decisions from the same transformer
10	Scaling up	fp16, LoRA and two GPUs: training GPT-2 XL on free hardware
11	This card	Golem-One XL

Table 2: The path from an empty Go module to Golem-One XL.

3 Part I — Building a token-by-token model in Go

3.1 The architecture

Figure 1 shows Golem as built: the GPT-2 architecture, rebuilt part by part in Go.

The original diagram placed each LayerNorm *after* its sub-layer. That is the “post-LN” layout of the original transformer [1] and GPT-1 [2]. Golem places it *before* (“pre-LN”), as GPT-2 does [3]. It was a deliberate change (Report 3), for two reasons:

- it trains more stably when many blocks are stacked;

- it is the only way OpenAI’s released GPT-2 weights, which Golem-One depends on, load unchanged.

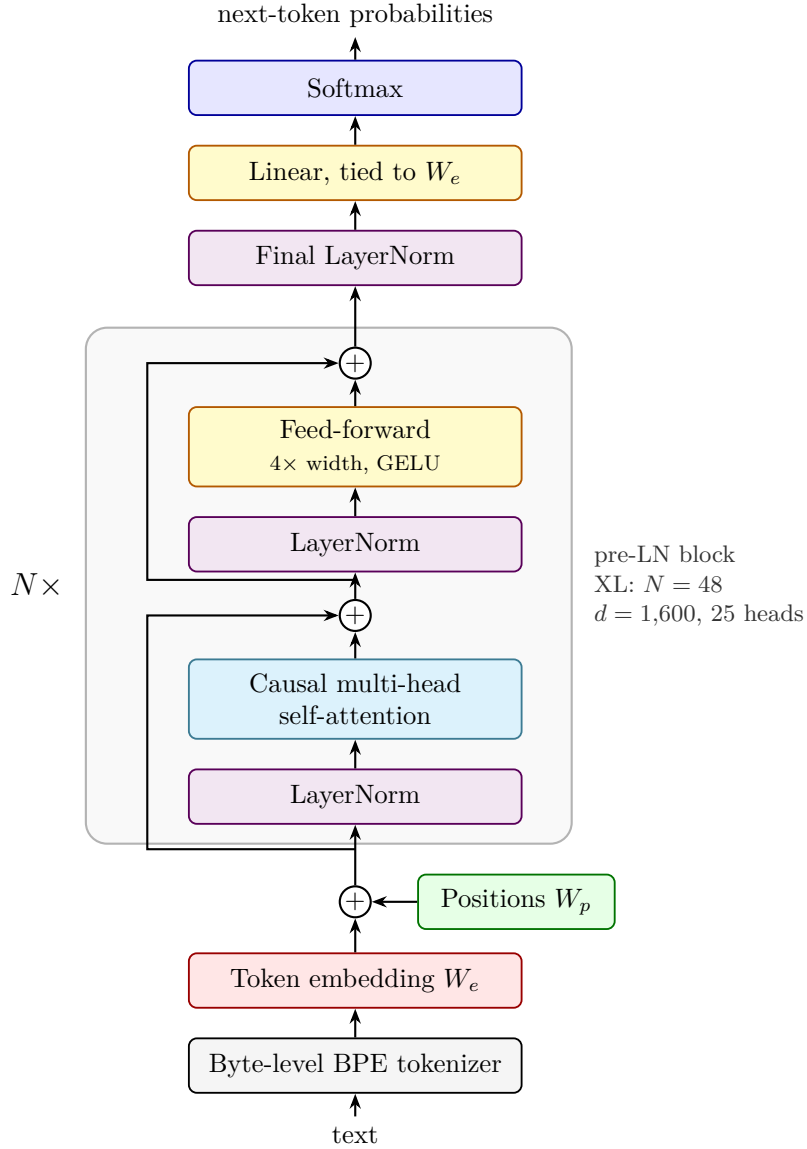


Figure 1: Golem’s transformer, the GPT-2 architecture rebuilt in Go. Each block normalizes, applies attention, and adds the result back to its input (the residual path on the left). It then does the same with the feed-forward network. The final layer turns the last hidden state into a probability for each of the 50,257 tokens.

3.2 Each part, the paper it follows, and how it was checked

The rule for every component

Every part below is plain Go in its own package. Each has a forward pass and a hand-derived backward pass, and is verified numerically:

- gradients against finite differences;
- speed-ups bit for bit against the naive loop;
- the finished model against OpenAI’s released weights and tokenizer.

Nothing is imported from a machine-learning library.

Tokenizer (`internal/tokenizer`). Text becomes integers through byte-level Byte Pair Encoding [4, 3]. Any string maps to known symbols because the base alphabet is the 256 bytes. The most frequent adjacent pairs are then merged, 50,000 times, into a vocabulary of 50,257 tokens. Golem’s tokenizer reproduces OpenAI’s `encoder.py` exactly, including its pre-tokenization pattern and byte-to-Unicode mapping.

Embeddings and positions (`internal/embedding`). Each token id selects a row of the embedding matrix $W_e \in \mathbb{R}^{V \times d}$, and a learned position table W_p is added: $h_0 = UW_e + W_p$, exactly as written in GPT-1 [2]. The fixed sinusoidal encoding of the original transformer [1] is available as an option.

Causal multi-head attention (`internal/attention`). Each token forms a query, key and value. Attention weights are $\text{softmax}(QK^\top / \sqrt{d_k} + M)$, where the mask M is $-\infty$ above the diagonal, so a token can only look at itself and earlier tokens [1]. The work is split across H heads that attend independently. A test checks that the mask is exact: changing a later token leaves every earlier output identical, bit for bit. Golem also computes attention in blocks, FlashAttention-style, so the full $T \times T$ matrix is never stored at once.

Layer normalization and residuals (`internal/nn`, `internal/block`). LayerNorm [5] rescales each token vector to zero mean and unit variance, then applies a learned gain and bias. Each sub-layer is wrapped as $x + f(\text{LN}(x))$. Report 3 measured why this matters: in a 12-block stack without normalization, attention barely changed the token vectors, while with pre-LN every sub-layer contributes.

Feed-forward network (`internal/ffn`). A two-layer perceptron applied to each position: it expands $4\times$, applies GELU [6] in the tanh form used by OpenAI’s code, and projects back. It holds two thirds of each block’s parameters. With it, twelve blocks plus embeddings total **124,439,808** parameters, exactly GPT-2 small.

Output head, softmax and loss (`internal/golem`). After the last block and a final LayerNorm, a linear layer produces one score per vocabulary token. Its matrix is *tied* to W_e , the same matrix that embeds tokens [7]. Softmax turns the scores into probabilities, and cross-entropy against the true next token is the training loss. Residual projections are initialized scaled by $1/\sqrt{2N}$, following GPT-2. An untrained model starts at the loss that theory predicts (6.12 nats), and a gradient check through the entire network confirms the backward pass.

Optimizer (`internal/optim`). AdamW [8], Adam with decoupled weight decay, with linear warmup and cosine decay.

Loading GPT-2 (`internal/safetensors`, `internal/golem`). Pre-training a model of this size from scratch needs far more compute than one person has. Golem therefore reads OpenAI’s released weights from their `safetensors` files into its own structures. All four GPT-2 sizes load straight from their `config.json`, and the parameter counts match exactly, from 124M to 1.56B.

3.3 Making it fast

A pure Go loop is slow, and training speed limits everything else. Report 6 describes the engine work:

- a 4×4 register-tiled Go kernel;
- an ARM64 NEON micro-kernel written in Go assembly, which keeps 96 running sums in vector registers and reaches **112 GFLOP/s per core**, about 86% of the chip’s peak;

- BLAS-style packing of operands;
- attention computed on the same kernel.

Fine-tuning went from 109 to about 253 tokens/s on an Apple M3 Pro, and every result stayed bit-for-bit identical to the naive loop.

A second engine, in CUDA and modeled on `llm.c` [18], keeps the whole model resident on an NVIDIA GPU. Report 10 adds four things to it:

- fp16 tensor cores with loss scaling [16];
- LoRA adapters [17];
- an options-only loss;
- two-GPU data parallelism.

3.4 Teaching it to converse

With the architecture complete, Golem learned to hold a conversation. It followed the second half of the InstructGPT recipe [9], with both stages implemented from scratch in Go (Report 7):

- **supervised fine-tuning** on human conversations from OpenAssistant [11], with the loss on the assistant’s replies only;
- **Direct Preference Optimization** [10] on pairs of replies that people ranked.

Fine-tuning lowered the held-out conversation loss from 2.484 to 2.163 and turned rambling web-text continuations into on-topic answers. At this point Golem was a complete token-by-token assistant.

4 Part II — From token by token to System One

4.1 Why generating text is a poor way to decide

A token-by-token model defines $p(y \mid x) = \prod_t p(y_t \mid x, y_{<t})$ and answers by sampling one token after another. Much of what software needs from a model, however, is a *decision*: which team gets this ticket, is this message spam, how urgent is it, which way should the snake move. Asking a generator to decide has five costs:

1. **Sequential passes**: one full pass per output token, each waiting for the last.
2. **A 50,257-way output layer at every step**, though only a handful of answers are legal.
3. **No validity guarantee**: the answer may be off-list, malformed, or a sentence instead of a label.
4. **Lost uncertainty**: the program sees one sampled answer, not how likely the alternatives were.
5. **Repeated work**: three questions about one text read the text three times.

4.2 Where the idea came from

TypeSafe AI’s **Jev** [12, 13, 14] made a different interface explicit and called it a **System One** model, after the fast, intuitive “system 1” of dual-process psychology [15]. You declare a state and typed questions with their allowed answers. The model returns a calibrated probability for every allowed answer, for all questions at once, and never an answer outside the list.

We adopted that *interface and its guarantees*. We did not adopt Jev’s model, which is described as a new non-autoregressive network trained with its own reinforcement-learning method, and whose weights are not public. We asked instead whether Golem’s existing transformer, unchanged, could provide the same guarantees if it were *read* differently. It can.

4.3 Golem-One: the same network, read differently

Typed questions. A request is a state (any text) and named questions. Each question has 2 to 255 options of one of three types:

- **YesNo:** returns $p(\text{yes})$ and $p(\text{no})$;
- **Choice:** named options, each with an optional description;
- **Score:** ordered levels, which also return the expected level.

Every answer carries its full distribution, the chosen option, and a confidence $c = 1 - H(p)/\log K$. The confidence is 0 when the model is torn and 1 when it is certain.

A label per option. The prompt lists the options under labels “A”, “B”, “C”, ... (numbers beyond 26). Each label is a single GPT-2 token, and the prompt ends in “**Answer:**”.

Read the state once, answer every question in parallel. The state fills the attention cache once. All questions are then appended in a single pass as independent *segments*. Each question attends to the whole state and to its own tokens, never to another question (Figure 2). Each is numbered as if it alone followed the state, so its result equals a separate pass over “state + that question”; tests confirm this to 10^{-4} .

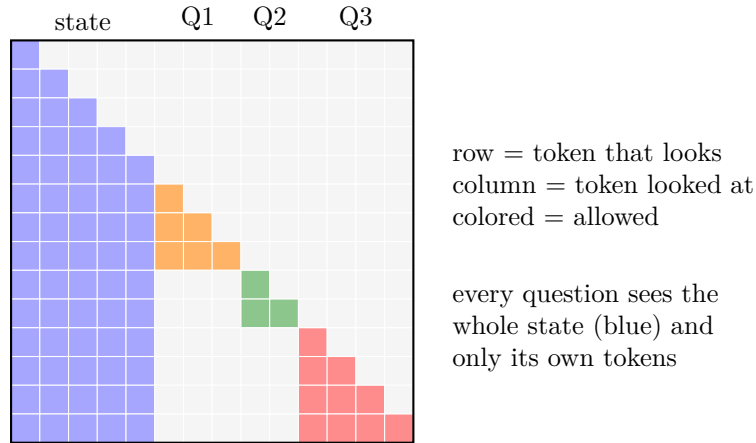


Figure 2: The segment mask for a 5-token state and three questions. The state is read once; the questions run together but never see each other.

Read only the allowed answers. At each question’s “**Answer:**” position, a generator would score all 50,257 tokens. Golem-One scores only the K label tokens and renormalizes:

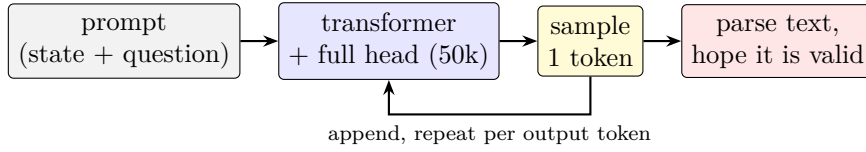
$$p(k) = \frac{\exp(e_{\ell_k}^\top h/T)}{\sum_{j=1}^K \exp(e_{\ell_j}^\top h/T)}. \quad (1)$$

Here h is the final hidden state, e_ℓ the tied embedding row of label ℓ , and T a calibration temperature.

Validity by construction

Probability exists only for declared options, so an invalid answer is impossible whatever the weights are. Nothing is generated, so there is nothing to parse. Training makes the answers *correct*; the construction already makes them *valid*.

Token-by-token Golem



Golem-One

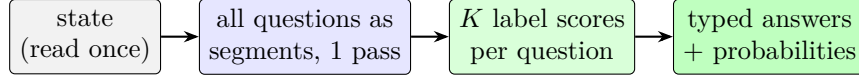


Figure 3: The same transformer used two ways.

	Token-by-token Golem	Golem-One
Output	free text	a probability for each declared option
Passes	one per generated token	the state once, then one pass for all questions
Output layer	50,257 rows every step	K label rows
Validity	must be parsed; can be off-list	guaranteed by construction
Randomness	sampled	none: the distribution is returned
Uncertainty	lost	full distribution + confidence
Training target	every token of the reply	the correct option's label
Network	Golem transformer	<i>the same</i> transformer and weights

Table 3: What changed, and what did not.

4.4 Why read differently instead of building a new network

- **Pre-training is the asset.** Any new architecture throws away some of what GPT-2 learned from billions of words.
- **One network, every tool.** The tokenizer, kernels, GPU engine, checkpoints and server from Part I are reused unchanged. The same weights can still write text.
- **The guarantees need no new parameters.** The mask and the restricted readout provide them.

5 From GPT-2 XL to Golem-One XL: what we changed

Golem-One XL is not a new network. We took the GPT-2 XL architecture as OpenAI released it, rebuilt it in Go, changed it in five places, and trained it on decisions. Figure 4 puts the two side by side. Everything grey is exactly GPT-2 XL, with the same shapes and the same frozen weights. Everything green is the same layer with fine-tuned weights. Everything orange is something we changed.

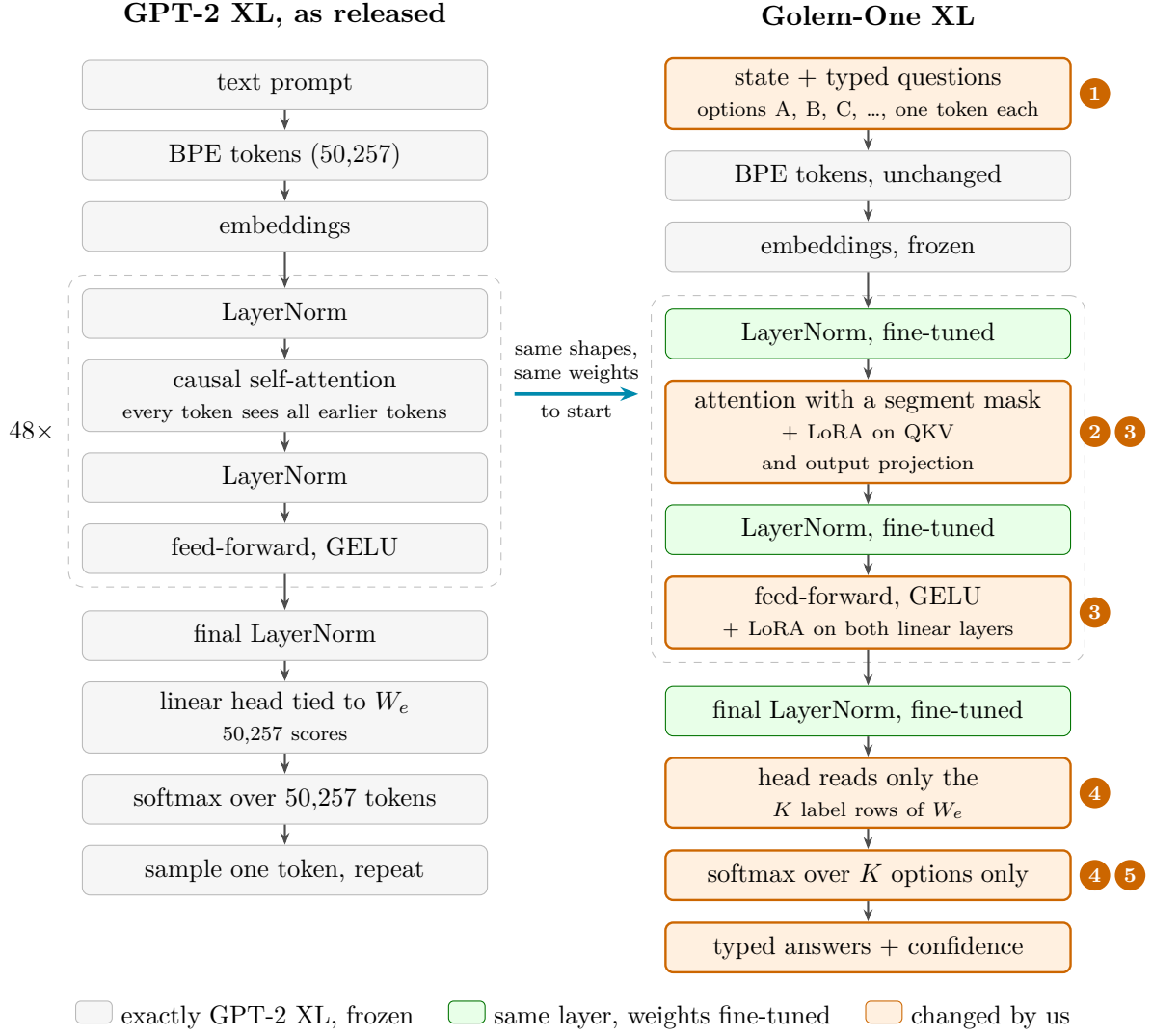


Figure 4: GPT-2 XL (left) and Golem-One XL (right). The numbered changes are described in Table 4. After training, the LoRA adapters are merged into the weights, so a Golem-One XL checkpoint has exactly GPT-2 XL’s 1,557,611,200 parameters.

	Change	GPT-2 XL	Golem-One XL
1	Input format	Any text; the model continues it.	A state and typed questions (YesNo, Choice, Score). Each option gets a label that is one GPT-2 token, and every question ends at “Answer:”.
2	Segment mask	A plain causal mask over one sequence.	The state is read once. All questions run in one pass as separate segments: each sees the whole state and itself, never another question, with positions renumbered to follow the state.
3	LoRA adapters	All 1.56B weights would need updating: about 25 GB, which does not fit on a free GPU.	The linear weights stay frozen in fp16. A rank-16 update $\frac{\alpha}{r}AB$ is learned on each of the four linear layers per block, together with the LayerNorm and bias terms: 20.7M trained parameters. The adapters are merged into the weights afterwards.
4	Restricted readout	50,257 scores and a softmax over all of them, then sampling.	Only the K label rows of W_e are scored and renormalized, with a calibration temperature T . An answer outside the options is impossible.
5	Options-only loss	Cross-entropy over the whole vocabulary on every token.	Cross-entropy over the K options only, on the answer position: exactly the quantity that is served.

Table 4: The five changes that turn GPT-2 XL into Golem-One XL. Changes 1, 2 and 4 are how the network is read; 3 and 5 are how it is trained.

Then we trained it on decisions. Figure 5 shows the whole path from OpenAI’s released file to a served decision model. Part III describes each step.

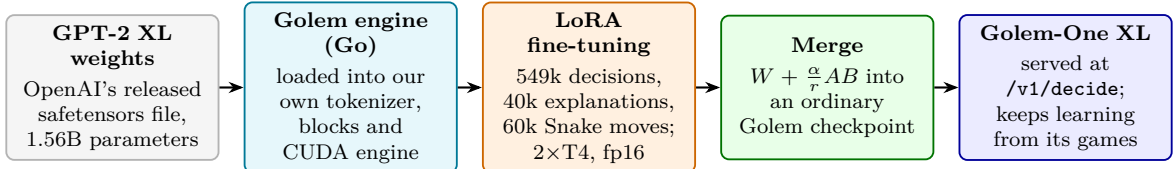


Figure 5: From released GPT-2 XL to Golem-One XL.

6 Part III — Training Golem-One XL

6.1 Why the largest size

At our scale, a fine-tuned model’s quality depends mostly on two things: the size of the pre-trained model and the amount of good data. We therefore chose GPT-2’s largest shape. Fully fine-tuning 1.56B parameters needs about 25 GB for weights and optimizer state, but a free Kaggle T4 has 15 GB. Report 10 closes that gap with four additions to Golem’s CUDA engine:

- **Mixed precision** [16]. Matrix products run in fp16 on tensor cores; weights, gradients and optimizer state stay fp32; dynamic loss scaling prevents underflow. It is 2.6–2.9× faster than fp32.
- **LoRA** [17]. Every linear weight is frozen and adapted by a rank-16 product $\frac{\alpha}{r}AB$. The frozen base is kept in fp16 only, so XL’s base takes 3.1 GB instead of 25 GB. Checkpoints merge the adapters and stay ordinary Golem checkpoints.

- **Options-only loss.** On decision rows the softmax runs over the answer labels only, exactly as the model is read. Training therefore optimizes the quantity that is served.
- **Two GPUs.** Micro-batches are split across both T4s, and gradients are summed before each identical update. Two-GPU runs are tested to match one-GPU runs.

With all four, XL fits in 14.6 GB per GPU and trains at about 1,600–2,200 tokens/s.

6.2 What it learns from

Skill	Open datasets	Questions
routing	Banking77 [27], CLINC150, HWU64, Bitext	55,000
classification	DBpedia, SMS Spam	24,982
moderation	Civil Comments, OpenAssistant message ratings [11]	33,064
entailment	MultiNLI, SNLI with e-SNLI explanations [21]	90,000
reading	BoolQ [28], PAWS, CosmosQA	62,227
commonsense	HellaSwag, PIQA, WinoGrande, CommonsenseQA with ECQA explanations	79,854
judgment	ETHICS	25,000
logic	ProofWriter, FOLIO	40,868
multi-step	StrategyQA	2,079
reasoning	BIG-bench	30,000
math	AQuA-RAT, MathQA (with worked rationales)	40,000
knowledge	ARC, OpenBookQA, QASC	16,461
many tasks	Super-NaturalInstructions [19]	50,114
games	Snake: positions from 300 expert games on a 10×10 board	60,000

Table 5: Training data. Every label is human-written or, for Snake, from a deterministic expert, and every source has an open license.

Four more task families are kept entirely out of training so that we can test questions of a kind the model has never seen:

- GoEmotions [29];
- MMLU [26];
- held-out BIG-bench Hard tasks;
- held-out Super-NaturalInstructions tasks.

Reading the options, not memorizing them. Option order is shuffled in every example, and tasks with many classes show the right answer among sampled distractors. The model cannot learn that “refund” is always “C”. Shuffling also removes the preference for the first option that the untrained model showed.

Learning why. Where a dataset gives human reasons, a second example teaches “*answer because explanation*”. This follows *Distilling Step-by-Step* [20]: learning to explain improves the answers. Golem-One never writes explanations when it decides, so they cost nothing at inference.

Snake: a game is just another decision. The board is drawn as a grid of characters (H head, S body, A apple, # wall) and becomes the state. The question is “Which way should the head move next?” with the options up, down, left and right. The answers come from an expert player. It takes the shortest path to the apple only when it can still reach its own tail afterwards; otherwise it follows its tail and waits. Snake is not a separate model. It is one of Golem-One’s skills, trained in the same run.

6.3 Training so far

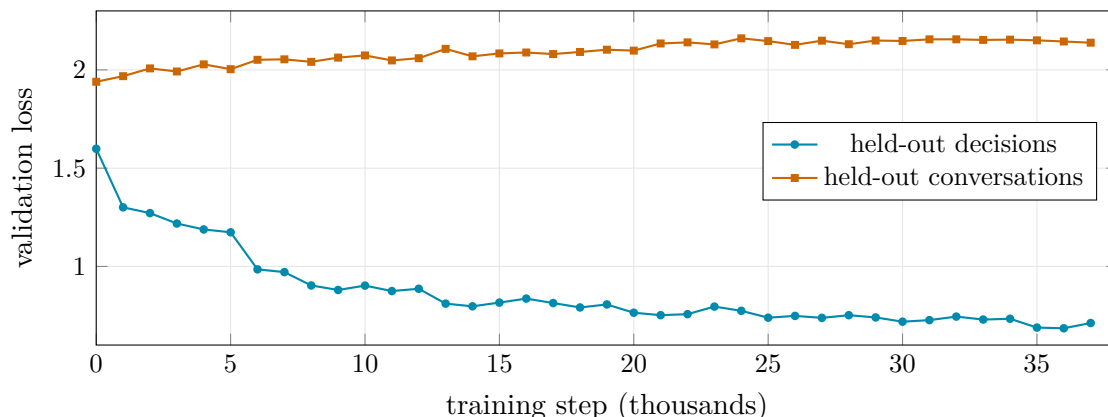


Figure 6: Validation losses every 1,000 steps. Step 0 is GPT-2 XL before any training.

The loss on 875 held-out decisions fell from **1.599** for untouched GPT-2 XL to **0.685** at step 36,000. On average, the probability the model gives the correct answer rose from about 20% to about 50%, across questions with very different numbers of options.

The loss on held-out conversations rose slightly, from 1.94 to 2.14. That is expected: this run spends all its compute on decisions and contains no conversation data.

6.4 Honest probabilities

A decision is only as useful as its probability is honest: when Golem-One says 80%, it should be right about 80% of the time. That is what lets software send uncertain cases to a person. The training loss is the log score, a *proper scoring rule* [24], which already rewards honest probabilities. After training we will measure:

- accuracy;
- log-likelihood;
- the Brier score [23];
- the expected calibration error [22].

We will then fit a single temperature T , which changes how sure the model says it is but never which answer it picks.

7 Learning from its own decisions

Golem-One can keep learning after training, from the outcomes of its own decisions. In the Golem-Snake demo, a browser plays games by asking Golem-One for each move and marks every move with a reward: +1 for an apple, -1 for a crash. The games go back to the server, which updates Golem's own weights with the policy gradient, REINFORCE with a baseline [25]. The update uses the same options-only loss, weighted by how much better than usual each decision turned out. A coach can also point out the best move and the moves that would have crashed at once. The learner is always Golem itself, never a separate small network.

8 Design choices

Question	Chosen	Set aside	Why
Foundation	Build Golem ourselves in Go; start from GPT-2 weights	A framework or an existing open model; pre-training from scratch	The point is to build and understand every part. Pre-training from scratch is out of reach for one person.
Block layout	Pre-LN (GPT-2)	Post-LN (original diagram, GPT-1)	Stable when deep; loads GPT-2’s weights.
Decisions	Segment mask + restricted readout	A new network (Jev); a separate classifier head	Keeps pre-training and text generation. Validity comes for free.
Labels	One token per option	Scoring multi-token option names	One position per question; no bias toward short names.
Loss	Softmax over the options only	Full 50k softmax	Trains exactly what is served.
Options	Shuffled, with distractors	Fixed order	Forces reading; removes first-option bias.
Size	XL, 1.56B	Medium, large	Pre-trained size is the strongest lever.
Training	LoRA + fp16 on two free T4s	Full fine-tuning	Full fine-tuning of XL does not fit in 15GB.
Data	28 open, human-labeled sources	One task; model-generated labels	Breadth for unseen questions; no inherited model errors.
Explanations	Learned as a side task	Chain-of-thought at inference	Better answers at no inference cost.
Learning	Golem updates itself through its API	A separate RL network	One model that decides and learns.

Table 6: The main choices behind Golem-One XL.

9 Intended use and limitations

Intended use. Decisions in software where the possible answers can be listed in advance:

- routing, tagging and triage;
- moderation hints;
- yes/no checks against a passage;
- multiple-choice reasoning;
- game moves.

Low-confidence answers are meant to go to a person.

Not for free-form answers, facts that must be true, or medical, legal, financial or other consequential decisions about people without human review. Golem-One XL is a research model from an independent learning project.

Limitations.

- **Not yet evaluated.** Accuracy, calibration and unseen-task results will be published when training finishes. This card reports training losses only.
- **Valid is not correct.** A confident wrong answer is still possible.
- **Inherited bias.** GPT-2’s web pre-training and the labeled datasets carry their own biases.
- **Weaker conversation.** The weights write text less well than Golem’s chat models.

- **Short training context.** Examples are at most 384 tokens, although the network accepts 1,024.
- **English only.**

10 Lineage: where each piece comes from

Golem component (Go package)	Research it follows	Report
Byte-level BPE (<code>tokenizer</code>)	Sennrich et al. [4]; GPT-2 [3]	1
Embeddings, positions (<code>embedding</code>)	GPT-1 [2]; Vaswani et al. [1]	1
Causal multi-head attention (<code>attention</code>)	Vaswani et al. [1]	2
LayerNorm, pre-LN block (<code>nn, block</code>)	Ba et al. [5]; GPT-2 [3]	3
Feed-forward with GELU (<code>ffn</code>)	GPT-1 [2]; Hendrycks & Gimpel [6]	4
Tied head, softmax, loss (<code>golem</code>)	Press & Wolf [7]; GPT-2 [3]	5
AdamW (<code>optim</code>)	Loshchilov & Hutter [8]	7
GPU engine (<code>gpu</code>)	llm.c [18]; mixed precision [16]	6, 10
Fine-tuning and DPO (<code>chat, dpo</code>)	InstructGPT [9]; DPO [10]	7
Typed decisions (<code>decide</code>)	Jev / System One [12, 15]	9
Label readout	Answer-letter scoring, as in MMLU [26]	9
LoRA adapters	Hu et al. [17]	10
Multi-task data, explanations	Super-NaturalInstructions [19]; Hsieh et al. [20]	11
Calibration	Guo et al. [22]; Brier [23]; Gneiting & Raftery [24]	9
Learning from outcomes (<code>decide.Learner</code>)	REINFORCE [25]	11

Table 7: Every Golem part, and the research it follows.

11 Using it

```
go-llm serve -decide-checkpoint golem-one-xl.bin -decide-weights models/gpt2-xl
curl -s localhost:8080/v1/decide -d '{
  "state": "I was charged twice for my order and want a refund.",
  "questions": {
    "category": {"type": "choice", "instructions": "What is this ticket about?",
      "options": ["billing", "bug", "delivery"]},
    "upset": {"type": "yes_no", "instructions": "Is the customer upset?"}}}'
```

References

- [1] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.
- [2] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever. *Improving Language Understanding by Generative Pre-Training*. OpenAI, 2018.
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.
- [4] R. Sennrich, B. Haddow, A. Birch. *Neural Machine Translation of Rare Words with Subword Units*. ACL, 2016.
- [5] J. L. Ba, J. R. Kiros, G. E. Hinton. *Layer Normalization*. arXiv:1607.06450, 2016.
- [6] D. Hendrycks, K. Gimpel. *Gaussian Error Linear Units (GELUs)*. arXiv:1606.08415, 2016.
- [7] O. Press, L. Wolf. *Using the Output Embedding to Improve Language Models*. EACL, 2017.
- [8] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization*. ICLR, 2019.

- [9] L. Ouyang et al. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS, 2022.
- [10] R. Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS, 2023.
- [11] A. Köpf et al. *OpenAssistant Conversations*. NeurIPS Datasets and Benchmarks, 2023.
- [12] TypeSafe AI. *Introducing System One Models & Jev*. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>, 2026.
- [13] TypeSafe AI. *Documentation: Introduction*. <https://docs.typesafe.ai/introduction>, 2026.
- [14] F. Copes. *A deep dive into Jev, TypeSafe’s System One model*. <https://flaviocopes.com/jev/>, 2026.
- [15] D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [16] P. Micikevicius et al. *Mixed Precision Training*. ICLR, 2018.
- [17] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
- [18] A. Karpathy. *llm.c: LLM training in simple, raw C/CUDA*. <https://github.com/karpathy/llm.c>, 2024.
- [19] Y. Wang et al. *Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks*. EMNLP, 2022.
- [20] C.-Y. Hsieh et al. *Distilling Step-by-Step!*. Findings of ACL, 2023.
- [21] O.-M. Camburu et al. *e-SNLI: Natural Language Inference with Natural Language Explanations*. NeurIPS, 2018.
- [22] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. *On Calibration of Modern Neural Networks*. ICML, 2017.
- [23] G. W. Brier. *Verification of Forecasts Expressed in Terms of Probability*. Monthly Weather Review, 1950.
- [24] T. Gneiting, A. E. Raftery. *Strictly Proper Scoring Rules, Prediction, and Estimation*. JASA, 2007.
- [25] R. J. Williams. *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*. Machine Learning, 1992.
- [26] D. Hendrycks et al. *Measuring Massive Multitask Language Understanding*. ICLR, 2021.
- [27] I. Casanueva et al. *Efficient Intent Detection with Dual Sentence Encoders*. NLP4ConvAI, 2020.
- [28] C. Clark et al. *BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions*. NAACL, 2019.
- [29] D. Demszky et al. *GoEmotions: A Dataset of Fine-Grained Emotions*. ACL, 2020.